

Lab 5

My code

Too much code in this project. Head over to my github repo where I supply example batch files along with all the project files.

<https://github.com/Qyoh7/CS162U>

AI Feedback

Overall Assessment

Your program correctly implements the requirements:

✓ Uses a **linked list** ✓ Maintains the list **ordered by title** ✓ Supports **search, add, remove, batch operations** ✓ Reads and writes a **binary file** ✓ Uses **dynamic memory correctly** ✓ Cleans up memory in the destructor

The overall structure using `Library` and `Song` classes is also good object-oriented design.

The program would likely **work correctly for most inputs**.

However, several improvements could make the code **cleaner, safer, and more maintainable**.

Strengths

1. Proper Linked List Implementation

Insertion keeps the list sorted:

```
while (cur != nullptr && cur->title < newSong->title)
```

This ensures the linked list remains **ordered by title**, which was a requirement.

2. Correct Memory Cleanup

Your destructor:

```
~Library()  
{  
    offWithHis(head);  
}
```

And helper:

```
while (cur != nullptr)
{
    Song* next = cur->next;
    delete cur;
    cur = next;
}
```

This properly prevents **memory leaks**.

3. Binary File Serialization

You correctly handled variable-length strings by writing the length first:

```
[length][characters]
```

This is the **correct approach for binary string serialization**.

4. Separation of Responsibilities

Your code is separated into:

- Song → data structure
- Library → linked list and file operations
- main → user interface

This is good **modular design**.

Issues and Suggested Improvements

1. Missing Header Guards

Your headers (Song.hpp , Library.hpp) lack include guards.

This can cause multiple-definition errors in larger programs.

Example fix:

```
#ifndef SONG_HPP
#define SONG_HPP
...
#endif
```

2. Song Should Not Expose next Publicly

Currently:

```
Song* next;
```

Public.

Better practice is making it **private**, with the `Library` class managing links.

3. Inefficient Duplicate Check

In `add()` you search for duplicates first:

```
while (cur != nullptr)
```

Then search again to insert.

This creates **two passes through the list**.

Better approach: detect duplicates **during insertion traversal**.

4. Constructor Loads File But Doesn't Reset songCount

You read:

```
fin.read(reinterpret_cast<char*>(&songCount), sizeof(size_t));
```

Then call `add()` which **increments songCount again**.

This means `songCount` becomes **double the actual value**.

You should instead:

```
read count
set count to 0
insert songs
```

5. Missing “unsaved changes” tracking

The assignment requires:

█ Prompt user to save if changes occurred before exiting.

Your code **does not track modifications**.

You need something like:

```
bool modified;
```

Set to true whenever:

- add
- remove
- batchAdd
- batchRemove

Then check on exit.

6. Batch Parser Is Fragile

Your batch parser assumes strict formatting:

```
title=...
artist=...
album=...
durationSeconds=...
```

If any field is missing or out of order the record breaks.

For coursework this may be acceptable, but a safer parser would validate fields.

7. Library.hpp Is Too Large

You implemented **all functions inside the header file**.

Best practice is:

```
Library.hpp  (declarations)
Library.cpp  (implementations)
```

Your instructor may not require this, but it's good practice.

Rewritten Design (Improved Structure)

Instead of rewriting all ~600 lines, here's what the **cleaner architecture would look like**.

Song

```

class Song
{
public:
    string title;
    string artist;
    string album;
    int durationSeconds;
    Song* next;

    Song(string t, string ar, string al, int d);
};

```

Library

Add:

```

bool modified;

```

Save Tracking

Example:

```

void add(...)
{
    ...
    modified = true;
}

```

Exit Handling

```

case 8:
    if (library.isModified())
    {
        cout << "Save changes? (y/n): ";
        ...
    }
    return 0;

```

Constructor Fix

Instead of:

```

fin.read(&songCount)
add(...)

```

Use:

```
size_t count;
fin.read(&count)

songCount = 0;

for(i < count)
    add(...)
```

Example Improved Add Logic

Single-pass version:

```
Song* cur = head;
Song* prev = nullptr;

while (cur && cur->title < title)
{
    prev = cur;
    cur = cur->next;
}

if (cur && cur->title == title)
{
    cur->artist = artist;
    cur->album = album;
    cur->durationSeconds = durationSeconds;
    return;
}

Song* newSong = new Song(title, artist, album, durationSeconds);
```

This avoids the double traversal.

Reflection You Can Submit

Here's something natural you can paste into your assignment.

The AI evaluation pointed out several areas where my program could be improved. While the program correctly implemented the linked list and binary file requirements, the AI noted that some design improvements could make the code safer and more maintainable. For example, the program did not track whether changes had been made since the last save, which is required by the assignment. It also pointed out that my `add()` function traversed the list twice when it could be optimized to perform insertion and duplicate detection in a single pass. Additionally, the AI suggested improvements such as adding header guards, improving file parsing robustness, and separating implementation code from header files. The rewritten design maintains the same functionality but improves structure and efficiency.

Honest Assessment

For a linked list + binary I/O assignment:

Your submission is **better than average**.

Typical student mistakes you avoided:

- memory leaks
- broken insertion
- incorrect binary serialization
- unordered list
- forgetting destructor
- incorrect string writing

The main missing feature relative to the assignment is:

unsaved-change detection before exit.

Everything else is **mostly polish**.

If you want, I can also show you the **3–4 things professors often secretly grade linked list projects on**, because they rarely write them in the assignment but still deduct points for them.